

Järgnevad skeemid kirjeldavad HK toimimist, tuginedes Shiny R rakenduste abil loodud piiratud võimekusega demoversioonile. See näitab tõhusalt, millised on detailsel tasemel reaalsed interaktsioonid süsteemi komponentide vahel

```
graph LR; A[Lõpetab uue teema õppimise] --> D[1. Tellib hindamise]; B[Käs on aeg plaanitud hindamiseks] --> D; C[Ei oska piiritleda oma õpivajadust] --> D; D --> E[2. Täidab testi]; E --> F[3. Loeb tagasisidet]; F --> G[4. Plaanib õppimist];
```

```
graph TD; OR[Õpirada (OR)  
(Süsteemi kese / AI agent)] -- "1. Hindamise päring  
(Teadmusgraafi objektid)" --> HK[Hindamiskomponent (HK)  
(ATA, TP, YP, TPn)  
[Application Service]]; HK -- "2. Tulemusprofiil  
(Raja korrigeerimiseks)" -.-> OR; HK -- "3. Testi esitamine vastamiseks  
4. Tulemuste kuva" --> UI[OR UI  
(Õpiraja Kasutajaliides)  
[Application Interface]]; style OR fill:#e0ffff,stroke:#000,stroke-width:1px; style HK fill:#e0ffff,stroke:#000,stroke-width:1px; style UI fill:#e0ffff,stroke:#000,stroke-width:1px;
```

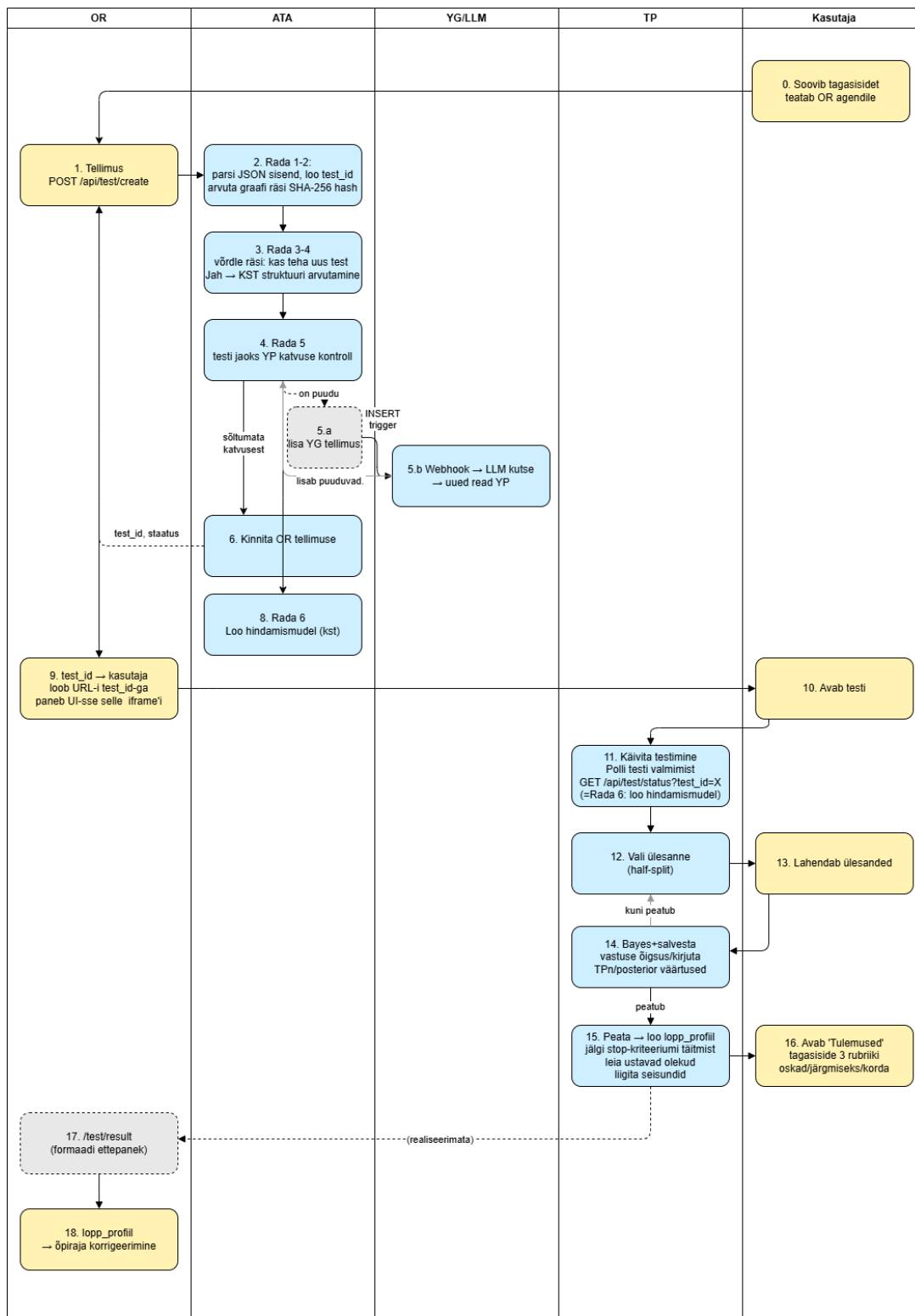
The diagram illustrates the OR system architecture, showing the interaction between three main components: the OR agent, the HK component, and the OR UI.

- Õpirada (OR) (Süsteemi kese / AI agent)**: The central AI agent, represented by a light blue box with a document icon.
- Hindamiskomponent (HK) (ATA, TP, YP, TPn) [Application Service]**: The evaluation component, represented by a light blue box with a circle icon.
- OR UI (Õpiraja Kasutajaliides) [Application Interface]**: The user interface, represented by a light blue box with a key icon.

The interaction flow is as follows:

- 1. Hindamise päring (Teadmusgraafi objektid)**: The OR agent sends a query to the HK component.
- 2. Tulemusprofiil (Raja korrigeerimiseks)**: The HK component returns a result profile to the OR agent (indicated by a dashed line).
- 3. Testi esitamine vastamiseks** and **4. Tulemuste kuva**: The HK component sends test data and results to the OR UI.

Protsessi vaade

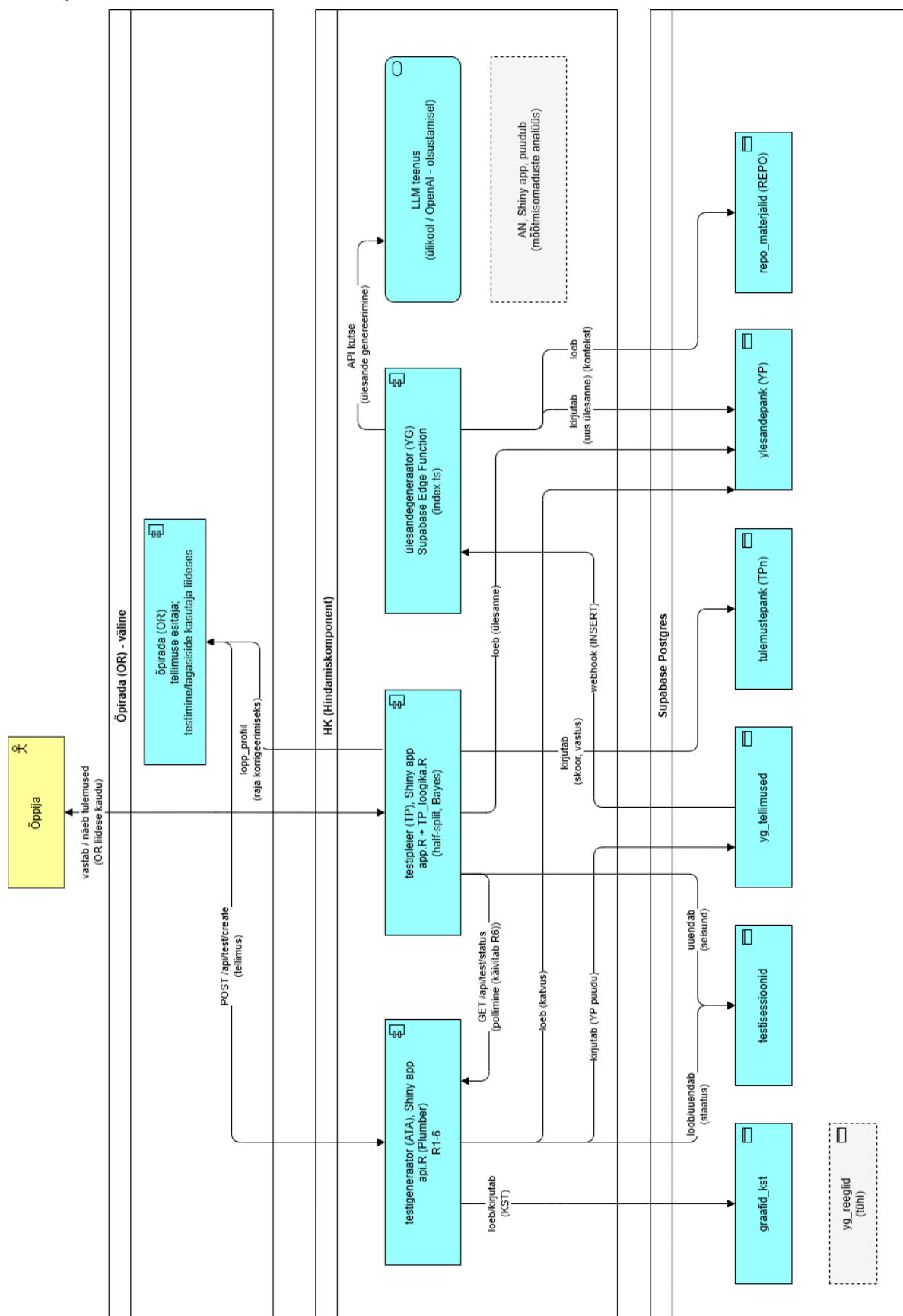


Märkused

OR - õpirajateenus, millega kasutaja saab valida ja eesmärgistada enda tegevusi õppimisel.
ATA - testi koostaja, võtab OR tellimuse ja disainib testi.
YG - ülesandegeneraator, loob testides vajalikud ülesanded.
TP - testi pleier, kuvab ülesanded kasutajale, järgides ATA nõudeid, skoorib, esitab tagasiside.
YP - ülesandepank, ülesannete ja nende meta-andmete hoidla.
TPn - tulemuste pank, ülesannete skooride hoidla.
Kasutaja - inimene, kes kasutab õppimisel OR, omades seal üks või enam õpirada.

Rada 1..n tähistab siin algses protsessi kirjelduses nimetatud samme.

Komponendi vaade



HOW to R

Dockeris vajalikud kihid:

Base image: rocker/r-ver:4.5.0 või rocker/shiny:4.5.0 kui Shiny enda võimalusi on vaja säilitada

Süsteemitasandi teegid apt-get install kaudu: libcurl4-openssl-dev, libssl-dev (httr jaoks, millel suhtlus Supabase'ga), libxml2-dev, uuid-dev (uuid jaoks, mis praegu annab test_id) + Build tööriistad (nt build-essentials)

R paketid ise: plumber, shiny, httr, jsonlite, kst, kstMatrix, digest, uuid. Need toovad pakette kaasa (nt kst – sets ja relations)

Keskkonnamuutujad, mis peavad tulema docker run -e või docker-compose.yml kaudu

Port + käivituskäsk (entrypoint). ATA_kst praegust kuju app.r vaja pole, plumber käivitub otse:

```
pr <- plumber::plumb("api.R") pr$run(host = "0.0.0.0", port = 8000)
```

R võimekuse (ühelõimelisuse piirang) laiendamine:

Horisontaalsne laiendamine

Käivitad mitu koopiat samast konteinerist (docker-compose scale ata=3 või Kubernetes/Docker Swarm replicas) + load balancer (Traefik, HAProxy, nginx)

```
# docker-compose.yml
services:
  ata:
    build: ./ata
    deploy:
      replicas: 3      # kolm R/Plumber protsessi paralleelselt
    environment:
      - SUPABASE_SERVICE_KEY=${SUPABASE_SERVICE_KEY}
    labels:
      - "traefik.http.routers.ata.rule=Host(`ata.example.com`)"
      - "traefik.http.services.ata.loadbalancer.server.port=8000"

  tp:
    build: ./tp
    deploy:
      replicas: 3
    environment:
      - SUPABASE_SERVICE_KEY=${SUPABASE_SERVICE_KEY}
      - ATA_BASE_URLS=http://ata:8000
    labels:
      - "traefik.http.routers.tp.rule=Host(`tp.example.com`)"
      - "traefik.http.services.tp.loadbalancer.server.port=3838"
```

```
# KRIITILINE TP jaoks – nn kleepuv sessioon, et kasutaja ei satuks keset testi teise protsessi (nt uue ülesande valiku hetkel):
```

- "traefik.http.services.tp.loadbalancer.sticky.cookie=true"
- "traefik.http.services.tp.loadbalancer.sticky.cookie.name=tp_session"

```
traefik:
```

```
image: traefik:v3.0
```

```
command: --providers.docker=true --entrypoints.web.address=:80
```

```
ports:
```

- "80:80"

```
volumes:
```

- /var/run/docker.sock:/var/run/docker.sock

Asünkroonne I/O orotsessi sees, paketid future ja promise

See oleks täitsa asjakohane HK koodi jaoks, kuna ATA/TP ei ole reeglina n-ö CPU-mahukad — need ootavad enamasti võrku (Supabase päringud, LLM kutsed).

future_promise({...}) võimaldab ühel R protsessil alustada uue päringu töötlemist, kui teine parasjagu HTTP-vastust ootab. Vähem mälu vaja kui ainult horisontaalsel. Efekt puuduks nt väga ebatavaliselt suure KST arvutuse puhul.

Katse osas vbl 2-3 koopia haldamine optimaalne.

ATA osas Plumber, api.R on olekuta – iga päring tegeleb otse Supabase'ga ega hoia midagi mälus. Päringuid saab piiranguteta mitme koopia vahel jagada.

TP (Shiny/ app.R) on olekuga ja iga kasutaja sessioon hoiab ühendust (websocket) ühe kindla R protsessiga kogu testi ajal. Sticky sessioon lubamine balancer'is koopiade puhul tähtis, et ühendus ei katkeks (vt märkus koodis).

Põhiline R-kood skriptis:

Räsi loomine, protsessi vaade, 2. samm

```
library(digest)

arvuta_graafi_hash <- function(solmed, seosed) {
  kanooniline <- list(
    solmed = sort(unique(solmed)),
    seosed = seosed[order(seosed[[1]], seosed[[2]]), , drop = FALSE]
  )
  digest(kanooniline, algo = "sha256")
}
```

KST struktuuri arvutamine, protsessi vaade, 3. samm

Pakett *kst* laseb testi moodustamiseks ja edasise testimise läbiviimiseks moodustada maatriksi (sõlmede ja seoste kirjeldus, mis sisuliselt on OR tellimuses olemas). Millegi tõttu teeb pakett seda puudulikult ning lahendamiseks tuleks kaasata teine pakett *relations*.

```
# 1. Teisendus: data.frame (seosed) -> endorelation objekt

seosed_suhtena <- endorelation(graph = set(
  do.call(rbind, lapply(seq_len(nrow(seosed)), function(i)
    tuple(seosed[i, 1], seosed[i, 2]))))
))

# 2. Sulund: lisa refleksiivsed + transitiivsed paarid, mida "seosed" ise ei sisalda

kvaasijarjestus <- reflexive_closure(transitive_closure(seosed_suhtena))

kstructure(kvaasijarjestus)
```

Tegelikult sai selle *kst* kitsenduse märkamisel maatriks koostatud koodis otse.

```
library(kst) # toob kaasa sets, relations

koosta_teadmusmudel <- function(metoodika, graaf_hash, solmed, seosed) {
  if (metoodika != "kst") stop("Ainult 'kst' realiseeritud")

  # 1. Siia paigutatud ka cache-kontroll - kas see graafi objektide hash on juba varem arvutatud?

  olemasolev <- sb_get(sprintf(
    "/graafid_kst?graaf_hash=eq.%s&select=graaf_hash,teadmusruum_maatriks", graaf_hash
  ))
  if (length(olemasolev) > 0 && nrow(olemasolev) > 0) {
    teadmusruum_raw <- fromJSON(olemasolev$teadmusruum_maatriks[[1]])
    return(list(uus = FALSE, teadmusruum = teadmusruum_raw, solmed = solmed))
  }

  # 2. Kehtivuse kontroll IGA võimaliku alamhulga jaoks - OTSE, mitte
  # kst::kstructure() suhte-tuletuse kaudu (vt allolev NB!)

  on_kehtiv_olek <- function(S, seosed) {
    if (nrow(seosed) == 0) return(TRUE)
    for (i in seq_len(nrow(seosed))) {
      eeldus <- seosed[i, 1]; solmus <- seosed[i, 2]
      if (solmus %in% S && !(eeldus %in% S)) return(FALSE)
    }
    TRUE
  }

  koik_alamhulgad <- unlist(
    lapply(0:length(solmed), function(k) combn(solmed, k, simplify = FALSE)),
    recursive = FALSE
  )
  kehtivad_olekud <- Filter(function(S) on_kehtiv_olek(S, seosed), koik_alamhulgad)
  struktuur <- kstructure(as.set(lapply(kehtivad_olekud, as.set)))
}
```

```

teadmusruum_raw <- lapply(as.list(struktuur), function(s) as.character(s))

# 3. Salvestamine (võidujooksu-kaitsega paralleelsete ATA replikate jaoks – kui seda kasutada)

sb_post("/graafid_kst", list(
  graaf_hash = graaf_hash,
  graafi_struktuur = toJSON(list(solmed = solmed, seosed = seosed), auto_unbox = TRUE),
  teadmusruum_maatriks = toJSON(teadmusruum_raw, auto_unbox = TRUE)
), prefer = "return=minimal,resolution=ignore-duplicates")

list(uus = TRUE, teadmusruum = teadmusruum_raw, solmed = solmed)
}

```

Selle ülesande saab ka Pythonis lahendada otse, kuigi soovitan järjepidevuse mõttes tugineda teadaoleva ajaloo ja publikatsioonidega R pakettidele.

Hindamismudeli loomine (kst), protsessi vaade, 8. samm

Ei kasuta ühtegi analüüsipaketti, ainult R base vahendid, mida NumPy saab array abil asendada.

```

koosta_hindamismudel <- function(metoodika, teadmusruum_raw, solmed, sõlme_parameetrid) {
  if (metoodika != "kst") stop("Ainult 'kst' realiseeritud")

  # 1. Olekute x sõlmede 0/1 maatriks

  K <- t(sapply(teadmusruum_raw, function(seisund) as.integer(solmed %in% seisund)))
  colnames(K) <- solmed
  n_seisundeid <- nrow(K)

  # 2. Ühtlane prior üle olekute

  P.K <- rep(1 / n_seisundeid, n_seisundeid)

  # 3. Sõlme-tasandi beta/eta (katvuskontrollist saadud ülesannete parameetrid)
  beta <- sapply(solmed, function(s) sõlme_parameetrid[[s]]$beta)
  eta <- sapply(solmed, function(s) sõlme_parameetrid[[s]]$eta)

  list(
    metoodika = "kst",
    K = K,
    P_K = setNames(P.K, apply(K, 1, function(r) paste(solmed[r == 1], collapse = ","))),
    solmed = solmed,
    beta = setNames(beta, solmed),
    eta = setNames(eta, solmed),
    yp_id = setNames(sapply(solmed, function(s) sõlme_parameetrid[[s]]$yp_id), solmed),
    ntotal = 0,
    koostatud = as.character(Sys.time())
  )
}

```

K maatriks — iga sammu (2) kehtiv olek muutetakse 0/1 reaks: 1, kui sõlm on selles olekus, 0 kui mitte. See ongi see maatriks, mida hiljem TP (sammud 10, 12-13) kasutab.

P_K — algne prior eeldus (enne ühtegi saadud vastust on iga olek võrdselt tõenäoline) — $1/(n_{olekud})$ igale olekule. Nimed (setNames) on olekute sõnalised kirjeldused (nt "A,B").

beta/eta — võetakse sammu "YP katvuse kontroll" tulemusest (sõlme_parameetrid, mis sisaldab iga sõlme jaoks valitud ülesande beeta_error/g_guess väärtusi). Praegu on kasutusel fiktiivsed vaikeväärtused (0.05/0.25), sest kalibreerimist pole seni tehtud.

yp_id — milline konkreetne ülesanne testis iga sõlme "esindab" selle mudeli raames.

ntotal, koostatud — metaandmed (kalibreerimiseks kavandatud, hetkel kasutuseta / ajatempel).

Testimisel ülesande adaptiivne valimine, protsessi vaade, 12. samm

Eelnevalt loodud testi mudel viitab ära kõik seisundite kirjeldamiseks vajalikud ülesanded. Tegelikult viiakse läbi adaptiivne testimine, mis püüab vähima ülesannete hulgaga välja peilida, milline on kasutaja teadmisseisund. Tegelikult hakkab TP esitama kasutajale ülesandeid ligikaudu teadmiseruumi keskkoha juurest. Sõltuvalt vastuste kehtivusest esitatakse edasi ülesandeid kas nõrgemate või tugevamate teadmisseisundite kohta (kui teeb ära, siis esitatakse raskem, kui ei tee ära, siis kergem ülesanne).

```
library(kstMatrix)

# a) Otsusta, mis sõlme kohta järgmisena küsida

vali_jargmine_solm <- function(posterior, K) {
  kmassesshalfsplit(posterior, K) # tagastab veeru-indeksi (sõlme)
}

# b) Vali sellele sõlmele konkreetne ja selles testis veel kasutamata ülesanne

vali_ylesanne_solmele <- function(solm, kysitud_yp_idd) {
  kandidaadid <- sb_get_q("/ylesandepank", list(
    graafi_objekt = paste0("eq.", solm), # AINULT sõlme järgi, mitte kursuse (vt "sõlme identiteedi põhimõte")
    staatus = "eq.kasutatav",
    select = "yp_id,beeta_error,g_guess,kasutamiste_arv"
  ))
  if (n_rida(kandidaadid) == 0) stop(sprintf("Sõlmele '%s' ei leitud ühtegi kasutatavat ülesannet.", solm))
  kasutamata <- kandidaadid[!(kandidaadid$yp_id %in% kysitud_yp_idd), ]
  valik <- if (nrow(kasutamata) > 0) kasutamata[1, ] else kandidaadid[1, ] # kõik juba küsitud - korda
  list(yp_id = valik$yp_id, beta = valik$beeta_error, eta = valik$g_guess,
    kasutamiste_arv = valik$kasutamiste_arv)
}

# c) Laadi kasutajale kuvamiseks valitud ülesande sisu YP-st

lae_ylesanne <- function(yp_id) {
  rida <- sb_get(sprintf("/ylesandepank?yp_id=eq.%s&select=*", yp_id))
}
```



```

rida <- rida[1, ]
list(
  yp_id = rida$yp_id, juhis = rida$juhis, tyvi = rida$tyvi,
  stiimul = if (length(rida$stiimul) == 0 || is.na(rida$stiimul)) NULL else rida$stiimul,
  voti = rida$voti,
  variandid = jarjesta_variandid(rida$voti, c(rida$distraktor_1, rida$distraktor_2, rida$distraktor_3))
)
}

```

Protseduur kulgeb:

- (a) kmassesshalfsplit() ütleb, milline SÕLM tuleb (nt "B") —puhas KST-arvutus ega puuduta andmebaasi.
- (b) kuna igal sõlmel on vähemalt 3 ülesannet YP-s, tuleb valida üks konkreetne, mida veel pole selles testis kasutatud (kysitud_yp_idd on andmed\$kytitud_yp_id-de loend).
- (c) viimasena laeb ülesande täisteksti kuvamiseks — sh järjestades valikud juhuslikult (tekst)/kahanevalt (arvud).

Tulemuste kontroll ja tõenäosused, protsessi vaade, 14. samm

Vastuste õigsuse ja vastava skoori (1-0) leidmine, tuleb salvestada tulemuste pank.

```

library(kstMatrix)
library(kmassessbayesian)

# a) Vastuse õigsuse kontroll + logi

vastus_oige <- identical(seisund$valitud_vastus, praegune$ylesanne$voti)
sb_post("/tulemustepank", list(
  test_id = andmed$test_id, yp_id = praegune$item$yp_id,
  skoor = as.integer(vastus_oige), valitud_vastus = seisund$valitud_vastus
))

# b) Bayes-uuendus - beta/eta TÄISVEKTORID (mudeli tasand), mitte üksuse skalaar

uuenda_posterior <- function(posterior, K, solm_indeks, vastus_oige, beta, eta) {
  kmassessbayesian(posterior, K, beta, eta, solm_indeks, as.integer(vastus_oige))
}
uus_posterior <- uuenda_posterior(
  andmed$posterior, andmed$K, praegune$solm_indeks, vastus_oige,
  andmed$testi_loogika$beta, andmed$testi_loogika$eta
)

# c) Salvesta uuendatud olek - unname() kohustuslik (vt eelmine viga)

uus_kysitud <- c(andmed$kytitud, list(list(
  yp_id = praegune$item$yp_id, solm = praegune$solm, vastus_oige = vastus_oige
)))
salvesta_tp_seisund <- function(test_id, posterior, kysitud) {
  sb_patch(sprintf("/testisessioonid?test_id=eq.%s", test_id), list(

```

```
    tp_seisund = toJSON(list(posterior = unname(posterior), kysitud = kysitud), auto_unbox = TRUE)
  ))
}
salvesta_tp_seisund(andmed$test_id, uus_posterior, uus_kysitud)
```

Protseduur kulgeb:

- (a) lihtne tekstivõrdlus aitab, kirjutab otse tulemuste panka (püsiv salvestus).
- (b) `kmassessbayesian()` võtab kogu K-maatriksi + mudeli-tasandi β/η + info, mis sõlme kohta küsiti ja hinnangu, kas vastati õigesti — ning tagastab uue tõenäosusjaotuse üle kõigi olekute.
- (c) salvestamine `tp_seisund`'ina — `unname()` on siin tähtis, kuna `kmassessbayesian()` tagastab posterior'i duplikaat-nimedega, mis JSON-serialiseerimisel muidu kokku varisevad.

Esitatud skeemid ja koodi kommentaarid ei käsitle praegu analüüsi komponenti (AN), millega analüüsida vastuseid tulemuste pangas ning korrigeerida ülesannete parameetreid ülesandepangas. See element lisandub edaspidi.